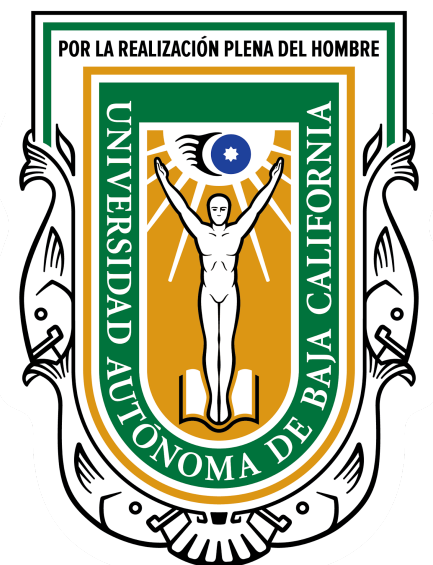


Universidad Autónoma de Baja California
Tijuana, Baja California, México



Facultad de Ciencias Químicas e Ingeniería

<i>Proyecto Final Sistemas Operativos</i>		
Materia:	Sistemas Operativos	
Docente:	Alma Leticia Palacios Guerrero	
Integrantes:	Baldenegro De la Vega José Angel Fernández Bañuelos Carlos Antonio	1288800 1289229
Fecha de act.:	05/06/2025	

ÍNDICE

Introducción	3
Scripts	3
cleanbin. Script para optimizar la limpieza de archivos temporales	3
memproc. Script para monitorear la cantidad de memoria que utiliza cada proceso del sistema	3
Comandos	4
unarch. Comando para concatenar 'n' cantidad de archivos a uno solo	4
divarch. Comando para separar un solo archivo en 'n' cantidad de archivos	4
newline. Comando para insertar una nueva línea de texto en una posición específica	4
Implementación de funcionalidades	5
Función cantidadpipes	5
Función cantidadargs	5
Función setDir	6
Función ejecutar_comando	7
Función mostrarHelp	8
Función shell	9
Función definepath	11
Función main	12
Manual de Usuario	13
Ejemplos con scripts propios	13
Ejemplos con comandos propios	15
Ejemplos con comandos del sistema	19
Conclusiones	20
Referencias:	21

Introducción

JShell es un shell personalizado que amplía las funcionalidades básicas de un intérprete de comandos tradicional, permitiendo la ejecución de comandos simples y encadenados mediante tuberías, además de manejar internamente comandos esenciales como `cd`. Para facilitar la ejecución de programas locales, JShell modifica la variable de entorno `PATH` incluyendo el directorio actual.

El shell incorpora scripts especializados como **cleanbin**, diseñado para liberar espacio de almacenamiento identificando y eliminando archivos temporales antiguos, y **memproc**, que permite monitorear en tiempo real el uso de memoria de los procesos, destacando con colores aquellos que superan un límite establecido.

Por otro lado, cuenta con comandos propios para trabajar con archivos de forma sencilla. Por ejemplo, **unarch** permite juntar varios archivos en uno solo, verificando que los archivos existan; **divarch** permite dividir un archivo en varios archivos más pequeños, repartiendo su contenido en partes lo más equilibradas posible según la cantidad de líneas que el usuario especifique; y **newline** permite insertar texto en una línea específica de un archivo, evitando que se generen espacios vacíos.

Scripts

cleanbin. Script para optimizar la limpieza de archivos temporales

Este script está diseñado para liberar espacio de almacenamiento del sistema al identificar los archivos con la extensión `.tmp` que no hayan sido modificados por 30 días o más. El script le permite al usuario visualizar todos los archivos que cumplan con las características anteriormente mencionadas y le ofrecerá tomar la decisión de eliminarlos o conservarlos. Si el usuario decide eliminarlos, se registrará en un archivo `.log` la eliminación, especificando la fecha, hora y los archivos que fueron eliminados.

memproc. Script para monitorear la cantidad de memoria que utiliza cada proceso del sistema

Este script le permite al usuario monitorear el uso de memoria RAM de cada proceso del sistema en tiempo real. El script le proporciona información cada 5 segundos al usuario, indicando con colores el uso de memoria de cada proceso. Si un proceso excede el límite establecido, este se ilustra de color rojo, de lo contrario el color utilizado para indicar la cantidad de memoria utilizada por un proceso es verde.

Comandos

unarch. Comando para concatenar 'n' cantidad de archivos a uno solo

Este comando tiene la necesidad de concatenar múltiples archivos proporcionados por el usuario y crear un solo archivo. Contiene validaciones que aseguran que el usuario solamente pueda ingresar archivos que existan dentro del sistema, es decir, si desea ingresar el nombre de un archivo que no existe, la acción no se realizará.

divarch. Comando para separar un solo archivo en 'n' cantidad de archivos

Con este comando, el usuario puede separar un archivo y dividir su contenido a través de diferentes archivos más pequeños, asegurando que el tamaño de todos los archivos sea lo más parecido posible. El usuario proporciona la cantidad de líneas de texto que en las que quiere separar el contenido del archivo original, por lo cual existen validaciones asegurando que el número de líneas ingresadas no sea mayor al total de líneas que contiene el archivo a dividir.

newline. Comando para insertar una nueva línea de texto en una posición específica

Con este comando, el usuario es capaz de insertar una nueva línea de texto en un archivo. El usuario selecciona el archivo y la línea en dónde quiere que se ingrese el nuevo texto, si la línea ingresada es mayor al total de líneas que contiene el archivo que desea modificar, se le notifica que se ingresará hasta el final del archivo, esto para evitar que el archivo se llene de líneas en blanco y solamente se ingrese el texto especificado.

Implementación de funcionalidades

Función cantidadpipes

```
int cantidadpipes(char *linea, char *cmds[])
{
    int count = 0;
    char *token = strtok(linea, "|");
    while (token != NULL && count < MAX_CMDS)
    {
        cmds[count++] = token;
        token = strtok(NULL, "|");
    }
    return count;
}
```

Imagen 1.1 - Función para calcular la cantidad de entubamientos

Se encarga de dividir una cadena de texto que contiene un comando completo ingresado por el usuario, separándolo en subcomandos utilizando el carácter de tubería (|) como delimitador. Para esto, usa la función strtok y almacena cada fragmento separado en el arreglo **cmds**, devolviendo la cantidad de comandos separados, lo cual se utiliza para manejar comandos encadenados mediante pipes.

Función cantidadargs

```
void cantidadargs(char *cmd, char *args[])
{
    int i = 0;
    args[i] = strtok(cmd, " ");
    while (args[i] != NULL && i < MAX_ARGS - 1)
    {
        args[++i] = strtok(NULL, " ");
    }
    args[i] = NULL;
}
```

Imagen 1.2 - Función para obtener los argumentos de un comando

Toma uno de los comandos individuales y lo divide en partes usando los espacios como delimitadores. Cada palabra del comando se guarda en un arreglo, y se deja un espacio al final para un apuntador NULL, que es necesario cuando se quiere ejecutar el comando con funciones como execvp. De esta forma, cada comando puede ejecutarse junto con sus parámetros correctamente.

Función setDir

```
int setDir(char **args)
{
    if (args[0] != NULL && !strcmp(args[0], "cd"))
    {
        char *dir = args[1];
        if (dir == NULL)
        {
            dir = getenv("HOME");
            if (dir == NULL)
            {
                printf("Error! HOME no definido\n");
                return 1;
            }
        }
        if (chdir(dir) != 0)
        {
            perror("cd");
        }
        return 1;
    }
    return 0;
}
```

Imagen 1.3 - Función para definir cd

Está diseñada para manejar internamente el comando cd. Cuando detecta que el comando ingresado es cd, verifica si se especificó una ruta; si no es así, toma la ruta por defecto que está en la variable de ambiente HOME. Usa la función chdir para cambiar el directorio y si falla imprime el error correspondiente.

Función ejecutar_comando

```

void ejecutar_comando(char *cmd, int prev_fd, int next_fd)
{
    char *args[MAX_ARGS];
    cantidadargs(cmd, args);
    if (setDir(args)) return;

    pid_t pid = fork();

    if (pid == 0)
    {
        if (prev_fd != -1)
        {
            dup2(prev_fd, STDIN_FILENO);
            close(prev_fd);
        }
        if (next_fd != -1)
        {
            dup2(next_fd, STDOUT_FILENO);
            close(next_fd);
        }

        execvp(args[0], args);
        perror("Fallo execvp");
        exit(EXIT_FAILURE);
    }
    else if (pid > 0)
    {
        waitpid(pid, NULL, 0);
    }
    else
    {
        perror("fork");
        exit(EXIT_FAILURE);
    }
}

```

Imagen 1.4 - Función para realizar la ejecución de comandos, y entubamientos.

Se encarga de ejecutar un comando individual, tomando en cuenta las redirecciones necesarias para manejar tuberías. Primero tokeniza el comando en argumentos y verifica si es un comando `cd` para tratarlo internamente. Luego crea un proceso hijo mediante `fork`. En el proceso hijo, redirige la entrada y salida estándar si corresponde para conectar con las tuberías previas y siguientes, y finalmente ejecuta el comando con `execvp`. En el proceso padre espera a que el hijo termine antes de continuar. En caso de error en `fork` o `execvp` se imprime un mensaje de error.

Función mostrarHelp

```
void mostrarHelp()  
{  
    printf("Comandos propios del shell:\n");  
    printf("c1 ---> Permite concatenar varios archivos a  
    printf("c2 ---> Permite separar un archivo en una ca  
    printf("c3 ---> Permite ingresar una nueva linea a c  
    printf("s1 ---> Permite monitorear la cantidad de me  
    printf("s2 ---> Permite borrar archivos temporales q  
}
```

Imagen 1.5 -Función para mostrar ayuda

La función simplemente muestra en pantalla un mensaje con los comandos personalizados disponibles en el shell, explicando brevemente qué hace cada uno, facilitando así la interacción para el usuario.

Función shell

```

void shell()
{
    char linea[MAX_LINE];
    char *cmds[MAX_CMDS];
    int status;

    while (1)
    {
        printf("JSHELL> ");
        fflush(stdout);

        if (fgets(linea, MAX_LINE, stdin) == NULL)
        {
            break;
        }
        linea[strcspn(linea, "\n")] = '\0';
        if (!strlen(linea)) continue;
        if (!strcmp(linea, "salir")) break;

        if(!strcmp(linea, "help") || !strcmp(linea, "ayuda"))
        {
            mostrarHelp();
            continue;
        }

        int cant = cantidadpipes(linea, cmds);
        int prev_fd = -1;
        int pipefd[2];

        for (int i = 0; i < cant; i++)
        {
            if (i < cant - 1)
            {
                if (pipe(pipefd) == -1)
                {
                    perror("pipe");
                    exit(EXIT_FAILURE);
                }
            }
            else
            {
                pipefd[1] = -1;
            }

            ejecutar_comando(cmds[i], prev_fd, pipefd[1]);

```

```
    if (prev_fd != -1)
    {
        close(prev_fd);
    }
    if (pipefd[1] != -1)
    {
        close(pipefd[1]);
    }

    prev_fd = pipefd[0];
}
}
```

Imagen 1.6 - Función para ejecutar el shell

La parte principal del programa está en la función shell, que se encarga de leer lo que el usuario escribe, entenderlo y ejecutar los comandos correspondientes. Funciona en un ciclo constante donde muestra un prompt, espera que el usuario ingrese una línea y atiende comandos especiales como salir o pedir ayuda. Luego, si el comando contiene tuberías, lo divide en partes para ir conectando la salida de un comando con la entrada del siguiente, creando así las conexiones necesarias entre ellos. Para cada parte, llama a la función encargada de ejecutarla, asegurándose de manejar bien las tuberías para que se abran y cierren en el momento justo, evitando problemas o desperdicio de recursos.

Función definepath

```

int definepath()
{
    char path[1024];
    if (getcwd(path, sizeof(path)) == NULL)
    {
        perror("No se encontro el directorio actual");
        return 0;
    }

    const char *pathactual = getenv("PATH");
    if (pathactual == NULL)
    {
        fprintf(stderr, "Error, no se encontro el PATH actual\n");
        return 0;
    }

    size_t tamanopath = strlen(pathactual) + strlen(path) + 2;
    char *nuevopath = malloc(tamanopath);
    if (nuevopath == NULL)
    {
        perror("Fallo de memoria al asignar nuevo path");
        return 0;
    }

    strcpy(nuevopath, pathactual);
    strcat(nuevopath, ":");
    strcat(nuevopath, path);

    if (setenv("PATH", nuevopath, 1) != 0)
    {
        perror("Fallo al poner el nuevo PATH");
        free(nuevopath);
        return 0;
    }

    free(nuevopath);
    return 1;
}

```

Imagen 1.7 - Función para definir path dentro del código

Antes de iniciar el shell, la función definepath se asegura de que el directorio actual se incluya en la variable PATH, lo que permite que el shell pueda ejecutar programas que estén en el mismo directorio desde donde fue invocado. Para ello obtiene la ruta actual, la concatena al valor existente de PATH y actualiza la variable. Esto facilita la ejecución de programas locales sin necesidad de rutas absolutas.

Función main

```
int main()
{
    if (!definepath()) return 0;
    shell();
    return 0;
}
```

Imagen 1.8 - Función main

Simplemente llama a **definepath** para preparar el entorno y luego inicia el ciclo principal del shell con la función **shell**. Si ocurre algún error al definir el PATH, el programa termina inmediatamente, de lo contrario el shell queda a la espera de comandos hasta que el usuario decida salir.

Manual de Usuario

Ejemplos con scripts propios

cleanbin

Se tienen inicialmente los siguientes archivos dentro del directorio HOME.

```
0 Jan  1 12:12 arch.tmp
0 Feb  2 23:21 archi.tmp
0 May  6 20:20 archiv.tmp
0 Jun  1 23:33 archivo.tmp
```

Imagen 2.1.1 - Archivos temporales en HOME.

El script solamente borrará aquellos que llevan más de 30 días sin sufrir alguna modificación:

```
JSHELL> cleanbin

SE ENCONTRARON LOS SIGUIENTES ARCHIVOS:
/home/so20251/jose_balde/archi.tmp
/home/so20251/jose_balde/arch.tmp
```

Imagen 2.1.2 - Ejecución del script cleanbin.

Posteriormente, al usuario se le presenta la opción de eliminar los archivos.

En caso de que el usuario no quiera eliminarlos, se termina la ejecución desplegando un mensaje indicando que la operación fue cancelada.

```
Desea eliminarlos? (s/n): n
Operacion cancelada
```

Imagen 2.1.3 - Confirmación para eliminar los archivos.

En caso de que el usuario quiera eliminarlos, la ejecución indica que han sido eliminados y se registran las eliminaciones dentro del archivo “**limpieza_archivos.log**”. Esto para llevar un control de lo que ha sido eliminado en caso de que el usuario necesite.

```
Desea eliminarlos? (s/n): s
Archivos eliminados correctamente
Registro guardado en /home/so20251/jose_balde/limpieza_archivos.log
```

Imagen 2.1.4 - Eliminando archivos temporales.

Archivo de registro.log

```
JShell> cat limpieza_archivos.log
[Thu Jun  5 20:51:11 UTC 2025] Archivos eliminados:
- /home/so20251/jose_balde/archi.tmp
- /home/so20251/jose_balde/arch.tmp
```

Imagen 2.1.5 - Evidencia de eliminación de archivos temporales.**memproc**

Para este script no se necesitan argumentos, solamente con ingresar en el intérprete de comandos el nombre del script se ejecutará y se podrá visualizar el uso de memoria de cada proceso del sistema, actualizándose cada 5 segundos.

```
Monitoreando uso de memoria por proceso
(actualiza cada 5 seg)
Alerta si algun proceso supera 100 MB
-----
-----
PID:NOMBRE:RSS(MB):ALERTA
599:mariadb:109:ALTO USO
26089:apache2:35:
26173:apache2:35:
26162:apache2:35:
26161:apache2:34:
234:systemd-journal:34:
26160:apache2:32:
26166:apache2:31:
```

Imagen 2.2 - Funcionalidad del script memproc

Ejemplos con comandos propios

unarch

Lo primero que podemos observar son los archivos a, b y c. Estos tienen en su contenido diferentes partes de la letra de una canción.

```
JSHELL> cat -n a
      1  Dale a tu cuerpo alegria
JSHELL> cat -n b
      1  macarena
JSHELL> cat -n c
      1  hey!
```

Imagen 3.1.1 - Archivos utilizados para demostración de unarch.

Entonces, para poder concatenar el contenido de los 3 archivos en un solo ingresamos los siguientes parámetros en la línea de comandos:

```
unarch <archivo_destino> <archivo_1> <archivo_2> [archivo_n]
```

```
JSHELL> unarch final c a b
Archivos concatenados exitosamente
```

Imagen 3.1.2 - Ejecutando el comando unarch.

Una vez ingresados los parámetros, continuamos con la ejecución tradicional, entonces presionamos la tecla ENTER para ejecutar el comando y nos proporciona con un mensaje indicando que la concatenación ha sido exitosa.

```
JSHELL> cat -n final
      1  Dale a tu cuerpo alegria
      2  macarena
      3  hey!
```

Imagen 3.1.3 - Demostración de funcionamiento.

Finalmente visualizamos el contenido del archivo destino, verificando que la concatenación en efecto fue exitosa y se realizó correctamente.

divarch

Inicialmente visualizamos el contenido del archivo que vamos a separar:

```
JSHELL> cat -n final
    1  Dale a tu cuerpo alegria
    2  macarena
    3  hey!
```

Imagen 3.2.1 - Archivo a dividir.

Posteriormente ingresamos el comando junto con sus parámetros para funcionar:

```
divarch <archivo_a_dividir> <número_líneas>
```

```
JSHELL> divarch final 1
Archivo dividido correctamente.
Nombres del archivo: final_01,02,etc
```

Imagen 3.2.2 - Ejecutando el comando divarch.

Mediante un mensaje se informa la división correcta de los archivos junto con los nombres de dichos archivos que fueron generados tras la división.

Finalmente, apreciamos el contenido de cada archivo para verificar que en efecto la división fue correcta.

```
JSHELL> cat -n final_01.txt
    1  Dale a tu cuerpo alegria
JSHELL> cat -n final_02.txt
    1  macarena
JSHELL> cat -n final_03.txt
    1  hey!
```

Imagen 3.2.3 - Demostración de funcionalidad.

newline

Primero, visualizamos el contenido del archivo al cuál queremos ingresar una nueva línea de texto.

```
JSHELL> cat -n archivo
1  hola
2  muy
3  buenos
4  días
```

Imagen 3.3.1 - Archivo para demostración.

Posteriormente ingresamos el comando junto con sus parámetros correspondientes:

```
newline <archivo> <número_línea> <texto>
```

Posteriormente, mediante unos mensajes, se informa que se está realizando la inserción de la nueva línea junto con el texto en el archivo. Finalmente indica que la acción fue realizada adecuadamente.

```
JSHELL> newline archivo 3 TREMENDOS
Insertando nueva línea en el archivo 'archivo'...
Linea insertada correctamente en el archivo 'archivo'.
```

Imagen 3.3.2 - Ejecución del comando newline.

Igual que con los otros comandos, visualizamos el contenido del archivo para verificar que todo haya sido realizado correctamente.

```
JSHELL> cat -n archivo
1  hola
2  muy
3  TREMENDOS
4  buenos
5  días
```

Imagen 3.3.3 - Archivo modificado

Ahora, ¿qué pasa si el usuario quiere ingresar una línea en una posición que excede el número de líneas del archivo?

Pues, sencillamente se le informa al usuario que el número que ingresó excede el tamaño de líneas del archivo, entonces simplemente el texto se ingresa al final del archivo.

```
JSHELL> newline archivo 100 AMIGOS
La línea 100 excede el numero de líneas (5) del archivo.
El texto sera insertado al final.
Insertando nueva línea en el archivo 'archivo'...
Línea insertada correctamente en el archivo 'archivo'.
```

Imagen 3.3.4 - Demostración de caso de prueba.

```
JSHELL> cat -n archivo
1  hola
2  muy
3  TREMENDOS
4  buenos
5  días
6  AMIGOS
```

Imagen 3.3.5 - Resultado final de la función.

Ejemplos con comandos del sistema

Estos son los comandos tradicionales del sistema, no se entra a mucho detalle debido a que son comandos sencillos de comprender y utilizar, solamente se demuestra que funcionan adecuadamente.

Comando ls

```
JSHELL> ls
arch          insertar.c.save
archivo      memproc
cleanbin     newline
concatenar   newline.c
concatenar.c shell
dividir      shell.c
dividir.c    shell.c.save
```

Imagen 4.1 - Ejecución del comando ls.

Comando pwd

```
JSHELL> pwd
/home/so20251/jose_balde/proyectoFinal
```

Imagen 4.2 - Ejecución del comando pwd.

Comando cd

```
JSHELL> cd
JSHELL> pwd
/home/so20251/jose_balde
```

Imagen 4.3 - Ejecución del comando cd

```
JSHELL> cd /home/so20251/jose_balde/proyectoFinal
JSHELL> pwd
/home/so20251/jose_balde/proyectoFinal
```

Imagen 4.4 - Ejemplo de comando cd + pwd.

Conclusiones

Baldenegro de la Vega Jose Angel:

Este proyecto me permitió desarrollar herramientas para la administración de tareas y gestión de archivos en un entorno Unix. Realizando scripts para facilitar la limpieza segura de archivos temporales antiguos y monitorear el uso de memoria utilizado por los procesos en tiempo real. Además de lograr realizar un comando para ingresar una nueva línea de texto en cualquier archivo mediante el uso de entubamiento para comunicar eficientemente los procesos.

Fernandez Bañuelos Carlos Antonio:

En este proyecto reforcé todos los conceptos básicos de Sistemas Operativos, aprendí sobre como simular un intérprete de comandos, como aplicar entubamiento, el funcionamiento e implementación de los archivos de bajo nivel y sus funcionalidades, se cumplieron todos los objetivos propuestos por nosotros.

Referencias:

Intérprete de comandos en C (Shell). (2022, 18 agosto). Jose Alberto Benítez Andrades.

<https://www.jabenitez.com/portfolio/interprete-de-comandos-en-c-shell/>

Equipo editorial de IONOS. (2022, 23 noviembre). *Pipes de Linux explicados*. IONOS Digital Guide.

<https://www.ionos.mx/digitalguide/servidores/configuracion/pipes-linux/#:~:text=En%20inform%C3%A1tica%2C%20un%20pipe%20es,como%20entrada%20para%20otro%20programa.>

Implementation of Shell using C · Dharak Kharod. (2014, 2 enero).

<https://dharakk.github.io/2014/01/02/introducing-pooles/>

Google Search. (s. f.).

https://www.google.com/search?q=que+es+un+proceso+hijo+en+sistemas+operativos&oq=que+es+un+proceso+hijo+en+sistemas+operativos&gs_lcrp=EgZjaHJvbWUyBggAEEUYOdIBCTE0MTY2ajBqNKgCALACAA&sourceid=chrome&ie=UTF-8

Pérez Ornelas, F., & Palacios Guerrero, A. L. Manual de prácticas: Taller de sistema operativo UNIX. Universidad Autónoma de Baja California, Facultad de Ciencias Químicas e Ingeniería.

Pipes in C. (2023). *CSC209H5: Software Tools & Systems Programming*.

<https://www.cs.toronto.edu/~rupert/209/lec09.pdf>